

Architecture Analysis and Interface Design

Student Club Event Ticketing and Budget Portal

Shishir Hegde | SRN PES1UG24CS438 | Section 5H

Comparison of the architectural styles

Style	Fit for this project	Main drawback
Layered	Separates browser pages, business rules and storage. A single transaction can reserve a budget or consume a ticket.	A shared database can become a bottleneck. Layers still depend on well-defined interfaces.
Microservices	Ticket validation and registration could be scaled separately if usage grew significantly.	Several deployments, network failures and cross-service consistency would add work to a small project.
Client-server	Central records help users see the same approval state, and students only need a browser.	This describes deployment more than internal organisation. One server remains a failure point.

Selection: a layered monolith with client-server access. These terms are compatible: layered describes the internal design, while client-server describes how the browser reaches it. There is no need for independently deployed services in this prototype.

Main design decisions

Budget requests advance through a fixed state sequence. A rejection ends that request. Budgets are reserved at submission, not after the Dean approves, because otherwise several pending requests could claim the same allocation. Students can register only after final approval.

A ticket contains a random identifier, event ID and attendee ID, protected by an HMAC signature. Check-in compares the signature and then changes an unused ticket to used in one transaction. The database remains the source of truth for expiry and ticket use.

Components and interfaces

Component	Provided interface	Responsibility
Authentication	IAuth	Validate login, maintain sessions and enforce account roles.
Event and Budget	IEventBudget	Create and edit drafts, reserve allocation, route approvals and keep the decision record.
Ticket Service	ITicket	Register attendees, enforce capacity and issue signed QR tickets.
Check-in Service	ICheckIn	Verify scanned text and consume valid tickets once.
Dashboard Service	IStatus	Return visible events, current states, registration counts and allocation.
SQLite Repository	IDataStore	Store users, clubs, proposals, items, decisions and tickets.
Portal Web UI	Browser pages	Require the five business interfaces through forms and JSON requests.

The component diagram has seven components, five UI-to-service interfaces and the shared data-access interface. Circles show provided interfaces; semicircles show required interfaces. Small boundary squares are ports. The diagram uses assembly connectors, which do not need arrowheads.

Implementation mapping

Logical component	Code location
Portal Web UI	5-Code/templates and 5-Code/static
Authentication	app.py: login, load_user and login_required
Event and Budget	services.py: create_event, submit and review
Ticket Service	services.py: register, token and ticket; app.py: ticket_svg
Check-in Service	services.py: checkin; app.py: checkin route
Dashboard Service	services.py: dashboard and details; app.py: status
SQLite Repository	services.py: db; schema.sql

These are logical components, not seven separately deployed programs. The small prototype keeps related business methods in one Portal class. A larger implementation could split that class into modules without changing the basic interfaces.

Data flow

Proposal: the lead submits a form; the service validates it; the repository stores the draft and budget items.

Approval: the reviewer posts a decision; the service checks the current stage; the repository records the sign-off and new state together.

Ticket: a student registers; the service checks approval and capacity; the repository creates one ticket; the UI displays its signed QR.

Entry: staff submit scanned text; the service verifies it; a conditional update marks the ticket used; the page displays acceptance or rejection.

Limits

The local server has no high-availability setup, university single sign-on or hardware scanner integration. Production would need HTTPS, managed credentials, backups, monitoring and a database/server setup measured under the expected load. No production deployment is claimed.